13. Optimize Cost While Maintaining Performance

Key Terms:

Term	Full Name	What it does
RI	Reserved Instance	Pre-committed compute for 1-3 years at discount
Spot Instance	Spare Capacity VM	Unused cloud capacity at 60-90% discount (can be evicted)
vCPU	Virtual CPU	One thread of a physical CPU core
SLA	Service Level Agreement	Guaranteed uptime/performance commitment
FinOps	Financial Operations	Practice of managing cloud costs
Auto-scaling	Automatic scaling	Adjusting resources based on demand

Problem Statement

Design strategies to optimize cloud costs in Azure while maintaining system performance and reliability.

Clarifying Questions to Ask

1. **Current spend**: What's the monthly cloud bill?
2. **Workload type**: Steady or variable traffic?
3. **Performance SLAs**: What latency/availability is required?
4. **Environment**: Prod/dev/test breakdown?
5. **Data requirements**: Hot vs. cold data ratio?
6. **Commitments**: Willing to commit for discounts?
7. **Flexibility**: Can workloads tolerate interruption?

High-Level Design


COST OPTIMIZATION STRATEGIES
1. RIGHT-SIZING
Before: D4v3 (4 vCPU, 16GB) @ $0.20/hr → 15% CPU avg
After: D2v3 (2 vCPU, 8GB) @ $0.10/hr → 30% CPU avg
Savings: 50%
2. RESERVED INSTANCES / SAVINGS PLANS
On-Demand: $100/month
1-Year RI: $70/month (30% savings)
3-Year RI: $50/month (50% savings)
Best for: Baseline, always-on workloads
3. SPOT INSTANCES
On-Demand: $0.20/hr
Spot: $0.04/hr (80% savings!)
Trade-off: Can be evicted with 30s notice
Best for: Batch processing, CI/CD, stateless workers
4. AUTO-SCALING
Traffic pattern:
6am 12pm 6pm 12am
Scale: 2 instances night → 10 instances peak
Without auto-scale: Pay for 10 instances 24/7
5. STORAGE TIERING
Hot: $0.018/GB (frequent access)
Cool: $0.010/GB (infrequent, 30+ days)
Archive: $0.002/GB (rare access, 180+ days)

```
|  |                                                                  |  |
|  |  Auto-tiering policy: Move to cool after 30 days                 |  |
|  |  └──────────────────────────────────────────────────────────────┘  |
└──────────────────────────────────────────────────────────────────────┘
```

API Design

Cost Management APIs

Method	Endpoint	Description
`GET`	`/api/v1/costs/summary`	Get cost breakdown by service
`GET`	`/api/v1/costs/forecast?period=30d`	Forecast future costs
`POST`	`/api/v1/budgets`	Create a budget alert
`GET`	`/api/v1/budgets/{budgetId}/status`	Get budget utilization

Performance Monitoring APIs

Method	Endpoint	Description
`GET`	`/api/v1/metrics/latency?service={name}&period=1h`	Get latency metrics
`GET`	`/api/v1/metrics/throughput`	Get throughput metrics
`GET`	`/api/v1/recommendations`	Get cost optimization recommendations
`POST`	`/api/v1/recommendations/{id}/apply`	Apply a recommendation

Scaling APIs

Method	Endpoint	Description
`PUT`	`/api/v1/autoscale/{service}/rules`	Configure autoscaling rules
`GET`	`/api/v1/autoscale/{service}/history`	Get scaling event history

Key Strategies

Strategy	Savings	Trade-off	Best For

Right-sizing	20-50%	May need scaling later	Overprovisioned VMs
Reserved Instances	30-50%	Upfront commitment	Steady workloads
Spot Instances	60-90%	Can be interrupted	Batch, CI/CD
Auto-scaling	30-70%	Complexity	Variable traffic
Storage tiering	50-80%	Access latency	Old data
Serverless	Variable	Cold start latency	Sporadic workloads

Implementation Checklist

```
IMMEDIATE WINS (Week 1):
□ Delete unused resources (orphaned disks, old snapshots)
□ Stop dev/test environments nights/weekends
□ Enable auto-shutdown for dev VMs
□ Review and remove unused IPs, load balancers

SHORT-TERM (Month 1):
□ Analyze utilization, right-size VMs
□ Move to reserved instances for baseline
□ Implement storage lifecycle policies
□ Use spot instances for batch jobs

LONG-TERM (Quarter):
□ Architect for serverless where appropriate
□ Implement auto-scaling policies
□ Multi-region optimization (use cheaper regions)
□ Establish FinOps practices and budgets
```

Monitoring and Governance

```
AZURE COST MANAGEMENT:
┌──────────────────────────────────────────────────────────────┐
│  1. Set budgets and alerts (80%, 90%, 100% thresholds)       │
│  2. Tag resources for cost allocation by team/project        │
│  3. Review Advisor recommendations weekly                     │
│  4. Use Cost Analysis for trend identification               │
│  5. Implement Azure Policy to prevent expensive resources    │
└──────────────────────────────────────────────────────────────┘

COST ALLOCATION TAGS:
  {
    "team": "platform",
    "project": "search",
    "environment": "production",
```

```
    "cost-center": "CC-1234"
  }
```

Trade-offs

Decision	Option A	Option B	Recommendation
Compute	On-demand	Reserved	**Mix** - RI for base, on-demand for burst
Availability	Multi-region	Single region	**Based on SLA** - multi costs 2x
Data retention	Keep all	Lifecycle rules	**Lifecycle** - archive old data
Instance type	Latest gen	Previous gen	**Latest** - often better price/perf

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
AKS/Kubernetes	Auto-scaling pods and spot instance management	Tech Guide
Redis	Caching layer to reduce compute and database costs	Tech Guide
CDN	Edge caching to reduce origin server load and bandwidth costs	Tech Guide

How to Explain It Well

> **Opening**: "Cost optimization balances spending with performance. I focus on right-sizing, reservations, and auto-scaling."

> **Key talking points**:
>
> - "Right-size first - most VMs run at 15% CPU, can downsize 50%"
> - "Reserved instances for baseline load - 30-50% savings"
> - "Spot instances for batch jobs - 80% savings, handle interruption"
> - "Auto-scale to match traffic - don't pay for night capacity during day"

> - "Storage tiering: hot for active, archive for compliance data"
